

APPENDIX

Appendix A Sơ đồ phân loại thuật toán Reinforcement Learning

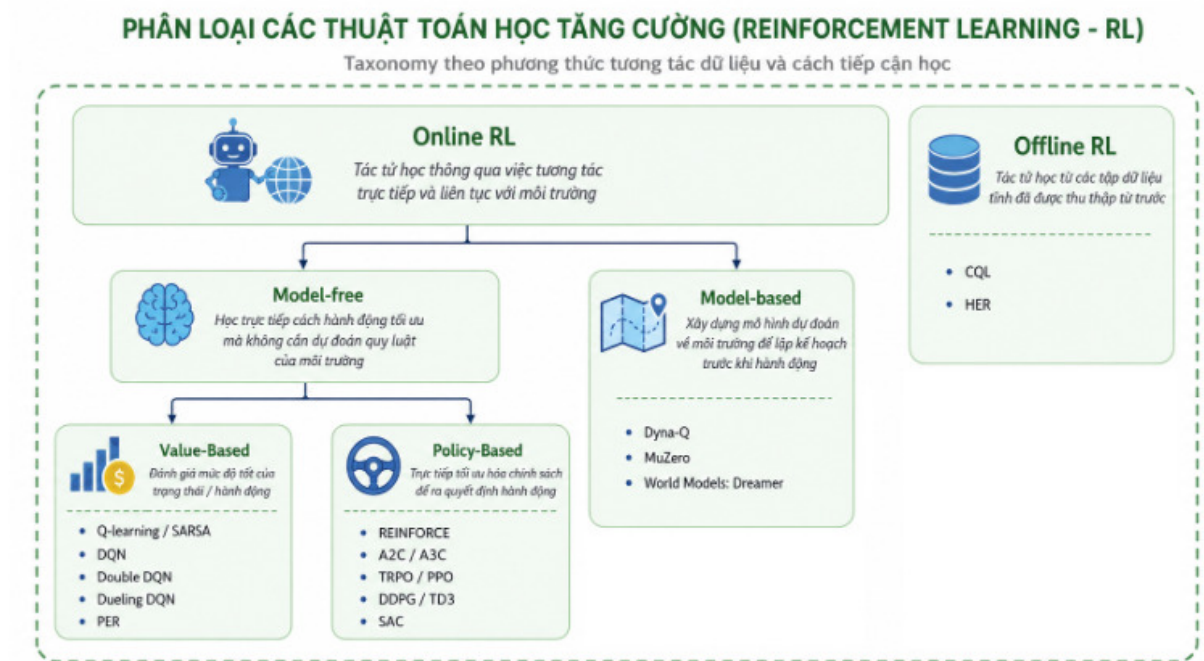


Figure A1: Sơ đồ phân loại các nhóm thuật toán Reinforcement Learning

A.1 Thuật toán dựa trên giá trị truyền thống

Q-Learning là thuật toán Off-Policy, cập nhật giá trị dựa trên hành động tốt nhất có thể ở trạng thái tiếp theo. Công thức cập nhật là:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Trong khi đó, SARSA là thuật toán On-Policy, cập nhật dựa trên hành động thực tế mà Agent chọn ở trạng thái kế tiếp:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

A.2 Học tăng cường sâu dựa trên giá trị

DQN mở rộng Q-Learning bằng cách dùng mạng neural để xấp xỉ hàm $Q(s, a)$, thay vì lưu trữ trong bảng Q-table. Điều này giúp thuật toán xử lý được các trạng thái phức tạp như hình ảnh hoặc dữ liệu cảm biến.

DQN sử dụng hai kỹ thuật quan trọng: Replay Buffer và Target Network. Replay Buffer lưu lại các trải nghiệm cũ và lấy mẫu ngẫu nhiên để huấn luyện, giúp giảm sự phụ thuộc tuần tự giữa các mẫu. Target Network giúp ổn định quá trình học bằng cách dùng một mạng riêng để tính giá trị mục tiêu.

Các biến thể quan trọng của DQN gồm Double DQN, Dueling DQN và Prioritized Experience Replay.

A.3 Policy Gradient và Actor-Critic

REINFORCE là thuật toán Policy Gradient cơ bản, tối ưu trực tiếp chính sách hành động $\pi_\theta(a|s)$ thay vì học Q-value. Nếu một hành động dẫn đến phần thưởng cao, xác suất chọn hành động đó sẽ tăng lên trong tương lai. Tuy nhiên, REINFORCE có nhược điểm là phương sai cao và học không ổn định.

Actor-Critic được phát triển để khắc phục hạn chế này. Trong đó, Actor chọn hành động, còn Critic đánh giá hành động hoặc trạng thái. A2C và A3C là hai biến thể phổ biến của Actor-Critic. A3C dùng nhiều worker cập nhật bất đồng bộ, còn A2C cập nhật đồng bộ nên dễ tận dụng GPU hơn.

PPO là một trong những thuật toán phổ biến nhất hiện nay. PPO giới hạn mức thay đổi của policy mới so với policy cũ bằng hàm clip, giúp quá trình học ổn định hơn nhưng vẫn tương đối đơn giản để triển khai.

A.4 Thuật toán cho điều khiển liên tục

Trong các bài toán như robot, xe tự hành hoặc điều khiển vật lý, hành động thường là giá trị liên tục, ví dụ như tốc độ, lực hoặc góc lái. DDPG, TD3 và SAC là các thuật toán tiêu biểu cho nhóm này.

DDPG dùng Actor để sinh hành động liên tục và Critic để đánh giá hành động đó. Tuy nhiên, DDPG dễ mất ổn định và nhạy với siêu tham số. TD3 cải tiến DDPG bằng cách dùng hai Critic, cập nhật Actor chậm hơn và thêm nhiễu vào hành động mục tiêu. SAC tiếp cận theo hướng Maximum Entropy, vừa tối đa hóa phần thưởng vừa duy trì sự ngẫu nhiên trong hành vi, giúp Agent khám phá môi trường tốt hơn.

A.5 Model-Based Reinforcement Learning và World Model

Model-Based Reinforcement Learning không chỉ học chính sách hành động mà còn học mô hình môi trường. Dyna-Q là thuật toán nền tảng, kết hợp giữa học từ trải nghiệm thật và lập kế hoạch từ trải nghiệm giả lập.

MuZero là một thuật toán hiện đại, không cần biết trước luật môi trường. Thay vào đó, nó học trạng thái ẩn, mô hình động lực và hàm dự đoán để phục vụ quá trình lập kế hoạch bằng Monte Carlo Tree Search.

Dreamer là một kiến trúc World Model, trong đó Agent học mô hình thế giới trong không gian latent. Sau đó, Actor và Critic được huấn luyện thông qua các rollout tưởng tượng trong latent space. Cách tiếp cận này giúp tăng hiệu quả sử dụng dữ liệu, đặc biệt trong các môi trường mà việc thu thập dữ liệu thật rất tốn kém như robot.

Appendix B Kết quả thực nghiệm các thuật toán reinforcement learning

Table B1: Kết quả benchmark khả năng tổng quát hóa giữa train map và test map-2

Thuật toán	Train WR	Test WR	Gen Gap	Avg Reward	Best	Avg Steps	Time (s)
REINFORCE	0.9%	0.0%	+0.9%	-12.85	+101.7	4.2	251.0
Q-Learning	37.1%	0.0%	+37.1%	+57.20	+110.8	9.3	497.4
SARSA	33.6%	0.0%	+33.6%	+53.21	+110.8	8.5	473.0
DQN	50.9%	0.0%	+50.9%	+57.86	+111.4	13.0	1210.0
Double DQN	41.7%	0.0%	+41.7%	+55.53	+111.0	12.6	1257.3
Dueling DQN	50.5%	0.0%	+50.5%	+55.57	+111.5	14.0	1306.3
PER	42.5%	0.0%	+42.5%	+54.54	+111.0	12.7	1588.8
A2C	45.3%	0.0%	+45.3%	+69.28	+111.3	8.9	555.8
PPO	4.7%	0.0%	+4.7%	-12.58	+107.6	43.8	1149.0
SAC	0.6%	0.0%	+0.6%	-14.24	+101.4	2.7	233.8

Ghi chú: Train WR cao nhưng Test WR thấp cho thấy mô hình có xu hướng overfitting vào map train và chưa tổng quát hóa tốt sang test map-2.

Bảng kết quả cho thấy các thuật toán có khả năng học khác nhau rõ rệt trên môi trường huấn luyện. Trong đó, DQN đạt Train Win Rate cao nhất với 50.9%, tiếp theo là Dueling DQN với 50.5%, A2C với 45.3%, PER với 42.5% và Double DQN với 41.7%. Điều này cho thấy nhóm thuật toán dựa trên Deep Q-Learning học được chính sách tương đối tốt hơn trong môi trường train. Về phần thưởng trung bình, A2C đạt Avg Reward cao nhất với +69.28, mặc dù Train WR chỉ đạt 45.3%. Điều này cho thấy A2C có thể tạo ra các hành vi ổn định hơn trong quá trình di chuyển, ít va chạm hoặc nhận được nhiều reward trung gian hơn. Trong khi đó, DQN có Avg Reward +57.86 và Best Reward +111.4, cho thấy thuật toán này có khả năng đạt được mục tiêu trong một số episode thành công. Các thuật toán như REINFORCE, PPO và SAC có kết quả thấp hơn rõ rệt. REINFORCE chỉ đạt 0.9% Train WR, PPO đạt 4.7%, còn SAC chỉ đạt 0.6%. Đồng thời Avg Reward của ba thuật toán này đều âm, cho thấy agent thường xuyên thất bại sớm, va vào tường, vật cản hoặc không tìm được đường đến goal. Một điểm đáng chú ý là Test WR của tất cả thuật toán đều bằng 0.0%. Điều này nghĩa là mặc dù một số thuật toán học khá tốt trên train map, nhưng không có thuật toán nào giải được test map. Do đó, Gen Gap của các thuật toán gần như bằng đúng Train WR. Ví dụ, DQN có Gen Gap +50.9%, Dueling DQN có Gen Gap +50.5%, A2C có Gen Gap +45.3%. Đây là dấu hiệu rất rõ của hiện tượng overfitting vào bản đồ huấn luyện.

Kết quả thực nghiệm cho thấy agent không chỉ học cách điều hướng tổng quát, mà nhiều khả năng đã học các đặc trưng riêng của train map. Mặc dù trong code đã có một số cơ chế giảm overfitting như random start position, random obstacle phase, exploration bonus, dropout, tăng entropy cho PPO/A2C và đánh giá định kỳ trên test map, nhưng kết quả Test WR vẫn bằng 0% cho tất cả thuật toán. Điều này chứng tỏ khả năng tổng quát hóa của agent vẫn còn rất hạn chế. Nhóm thuật toán DQN, Double DQN, Dueling DQN và PER cho kết quả train tốt hơn vì chúng có replay buffer và cơ chế học giá trị hành động tương đối phù hợp với môi trường rời rạc 4 hành động. Đặc biệt, DQN và Dueling DQN đạt Train WR khoảng 50%, cho thấy chúng học được một số chiến lược hiệu quả trong map huấn luyện. Tuy nhiên, khi chuyển sang test map có cấu trúc tường, vị trí start, goal và vật cản khác, chính sách đã học không còn hiệu quả. A2C là thuật toán đáng chú ý vì có Avg Reward cao nhất. Điều này có thể cho thấy A2C chưa chắc thắng nhiều nhất, nhưng hành vi trung bình của nó tốt hơn, ví dụ di chuyển được lâu hơn, tránh va chạm tốt hơn hoặc tiến gần goal hơn. Tuy nhiên, việc Test WR vẫn bằng 0% cho thấy A2C cũng chưa học được quy luật điều hướng có tính tổng quát. Ngược lại, REINFORCE, PPO và SAC cho kết quả kém trong thí nghiệm này. Với REINFORCE, nguyên nhân có thể đến từ phương sai gradient cao và reward sparse, khiến quá trình học không ổn định. PPO thường cần nhiều bước rollout, batch lớn và tuning kỹ hơn để hoạt động tốt. SAC trong phiên bản discrete đơn giản này cũng có thể chưa phù hợp hoặc chưa được thiết kế đầy đủ như SAC chuẩn với actor-critic riêng biệt. Về thời gian chạy, SAC và REINFORCE nhanh nhất, nhưng hiệu quả học rất thấp. DQN, Double DQN, Dueling DQN, PER và PPO mất nhiều thời gian hơn, đặc biệt PER và PPO, nhưng không tạo ra khả năng tổng quát hóa tốt hơn. Điều này cho thấy thời gian huấn luyện dài hơn không đồng nghĩa với policy tốt hơn nếu môi trường train chưa đủ đa dạng hoặc state representation chưa đủ mạnh.

Appendix C Các thành phần cơ bản của môi trường Reinforcement Learning

Table C2: Các thành phần cơ bản của môi trường Reinforcement Learning

Thành phần	Vai trò
State / Observation	Thông tin mà Agent quan sát được từ môi trường.
Action Space	Tập các hành động hợp lệ mà Agent có thể chọn.
Reward Function	Hàm đánh giá hành động của Agent là tốt hay xấu.
Transition Dynamics	Quy luật chuyển từ trạng thái hiện tại sang trạng thái tiếp theo.
Termination Condition	Điều kiện kết thúc một episode.
Info	Thông tin phụ dùng cho debug, đánh giá hoặc logging.

Bảng B1 mô tả cấu trúc tiêu chuẩn của một môi trường Học tăng cường (Reinforcement Learning) thông qua sáu thành phần định danh cốt lõi. Quá trình vận hành bắt đầu khi tác tử (Agent) tiếp nhận dữ liệu đầu vào (State/Observation) để trích xuất và thực thi một quyết định hợp lệ trong không gian hành động (Action Space). Ngay sau đó, môi trường sẽ phản hồi bằng cách cập nhật trạng thái mới theo quy luật chuyển đổi (Transition Dynamics), đồng thời trả về

một giá trị thông qua hàm phần thưởng (Reward Function) nhằm đánh giá mức độ hiệu quả của quyết định. Vòng lặp tương tác này duy trì liên tục cho đến khi hệ thống đạt tiêu chí dừng chu kỳ (Termination Condition), đi kèm với việc trích xuất các dữ liệu ngoại vi (Info) nhằm phục vụ công tác giám sát, đánh giá và lưu vết mô hình.

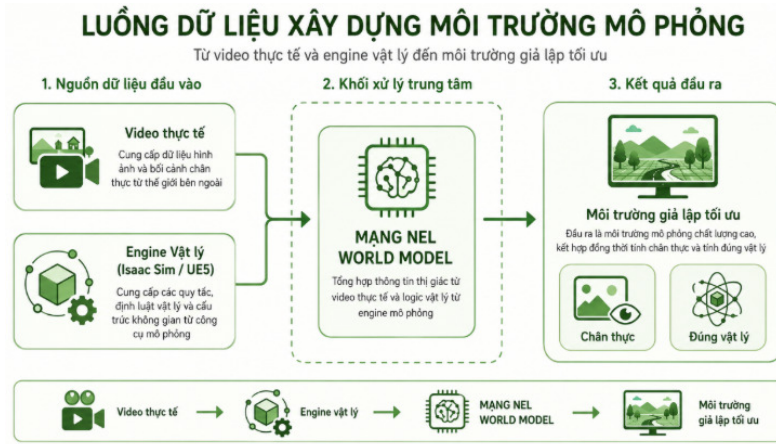
Appendix D Các kỹ thuật thiết kế Reward trong Reinforcement Learning

Bảng D3 tổng hợp một cách hệ thống các phương pháp thiết kế hàm phần thưởng (reward) trong Học tăng cường, cung cấp cái nhìn toàn diện về ý tưởng cốt lõi, ưu điểm và hạn chế của từng chiến lược. Cụ thể, các phương pháp cơ bản như Sparse Reward và Dense Reward thể hiện sự đánh đổi giữa việc định chuẩn chính xác mục tiêu cuối cùng (tránh sai lệch) và việc duy trì tốc độ hội tụ của mô hình. Để tối ưu hóa quá trình huấn luyện, các kỹ thuật định hình phần thưởng như Reward Shaping ($R' = R + F(s, a, s')$) và Potential-Based Shaping ($R' = R + \gamma\Phi(s') - \Phi(s)$) được sử dụng nhằm tăng tốc độ học nhưng vẫn bảo toàn tính toàn vẹn của chính sách tối ưu. Bên cạnh đó, Reward Decomposition ($R = w_1R_1 + w_2R_2 + \dots + w_nR_n$) hỗ trợ giải quyết các bài toán phức tạp bằng cách phân rã mục tiêu, dù đòi hỏi sự tinh chỉnh trọng số khắt khe. Nhằm xử lý vấn đề phần thưởng thưa thớt và rủi ro hội tụ sớm, báo cáo chỉ ra các giải pháp can thiệp nâng cao bao gồm Curriculum Reward (thiết lập lộ trình môi trường từ dễ đến khó), Intrinsic Reward (tạo động lực khám phá nội tại) và Entropy-Regularized Reward ($J = \mathbb{E}[R] + \alpha H(\pi)$). Cuối cùng, khía cạnh an toàn và triển khai thực tiễn được nhấn mạnh thông qua Constraint-Based Reward (tối ưu hóa kèm ràng buộc an toàn), cảnh báo về hiện tượng Reward Hacking (tác tử tối đa hóa điểm số nhưng sai nhiệm vụ), và đề xuất Nguyên tắc thực tế: khởi đầu với hàm phần thưởng tối giản, sau đó bổ sung tinh chỉnh có kiểm soát để giảm thiểu nguy cơ lệch mục tiêu trong quá trình phân tích.

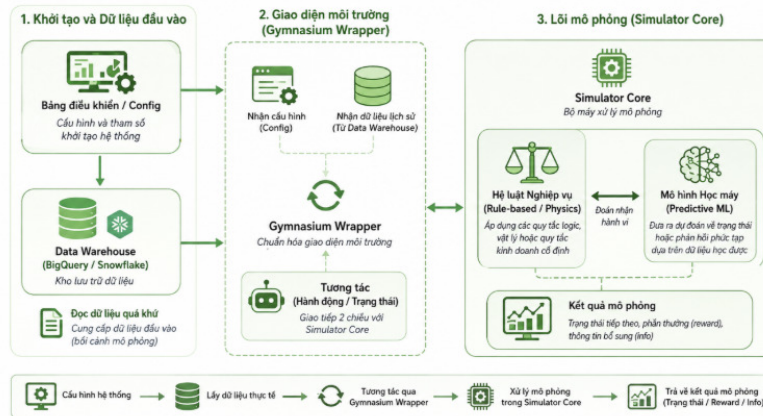
Table D3: Tóm tắt các tư duy thiết kế reward trong Reinforcement Learning

Phương pháp	Ý tưởng chính	Ưu điểm	Hạn chế / Lưu ý
Sparse Reward	Chỉ thưởng khi Agent đạt mục tiêu cuối.	Mục tiêu rõ ràng, ít gây sai lệch.	Học chậm, khó khám phá.
Dense Reward	Thưởng hoặc phạt ở nhiều bước trung gian.	Giúp Agent học nhanh hơn.	Dễ khiến Agent học sai hành vi mong muốn.
Reward Shaping	Thêm tín hiệu phụ vào reward gốc: $R' = R + F(s, a, s')$.	Tăng tốc quá trình học.	Reward phụ quá mạnh có thể làm lệch mục tiêu chính.
Potential-Based Shaping	Dùng hàm tiềm năng: $R' = R + \gamma\Phi(s') - \Phi(s)$.	Học nhanh hơn nhưng vẫn bảo toàn chính sách tối ưu.	Cần thiết kế hàm $\Phi(s)$ hợp lý.
Reward Decomposition	Tách reward thành nhiều phần: $R = w_1 R_1 + w_2 R_2 + \dots + w_n R_n$.	Biểu diễn được nhiều mục tiêu cùng lúc.	Khó chọn trọng số phù hợp.
Curriculum Reward	Thiết kế reward hoặc môi trường theo lộ trình từ dễ đến khó.	Giúp huấn luyện ổn định hơn.	Cần lộ trình tăng độ khó hợp lý.
Intrinsic Reward	Thưởng cho khám phá, novelty, curiosity hoặc prediction error.	Hữu ích trong môi trường sparse reward.	Có thể khiến Agent khám phá quá nhiều mà quên mục tiêu chính.
Entropy-Regularized Reward	Tối đa hóa cả reward và entropy: $J = \mathbb{E}[R] + \alpha H(\pi)$.	Tăng khám phá, tránh hội tụ quá sớm.	Cần chọn hệ số α phù hợp.
Constraint-Based Reward / Safe RL	Tách mục tiêu tối ưu và ràng buộc an toàn.	Phù hợp với robot, xe tự hành và tài chính.	Phức tạp hơn reward thông thường.
Reward Hacking	Agent tối đa hóa reward nhưng không làm đúng nhiệm vụ thật.	Giúp nhận diện lỗi thiết kế reward.	Cần kiểm tra hành vi thực tế, không chỉ nhìn tổng reward.
Nguyên tắc thực tế	Bắt đầu từ reward đơn giản, thêm shaping hoặc penalty có kiểm soát, dùng ablation study.	Giảm nguy cơ lệch mục tiêu, dễ phân tích.	Cần thử nghiệm nhiều cấu hình reward khác nhau.

Appendix E Quy trình thiết kế bài toán Reinforcement Learning



(a) Quy trình xây dựng môi trường Reinforcement Learning



(b) Quy trình sử dụng môi trường Reinforcement Learning để huấn luyện Agent

Figure E2: Hai quy trình chính trong thiết kế và sử dụng môi trường Reinforcement Learning

Bảng C2 trình bày quy trình hệ thống hóa gồm 8 bước để thiết kế một bài toán Học tăng cường (Reinforcement Learning).

- Giai đoạn nền tảng (Bước 1-3): Tập trung chuyển đổi mục tiêu thực tế thành bài toán quyết định tuần tự, đồng thời định nghĩa phạm vi dữ liệu quan sát (observation) và tập hành động (action space) hợp lệ cho tác tử.

Table E4: Quy trình thiết kế bài toán Reinforcement Learning

Bước	Nội dung thực hiện
1. Xác định bài toán	Chuyển mục tiêu kinh doanh thành bài toán ra quyết định tuần tự.
2. Thiết kế observation	Xác định dữ liệu nào Agent được phép quan sát.
3. Thiết kế action space	Xác định tập hành động mà Agent có thể thực hiện.
4. Thiết kế reward	Chuyển mục tiêu kinh doanh thành tín hiệu phần thưởng.
5. Xây dựng transition	Mô phỏng cách hệ thống thay đổi sau mỗi hành động.
6. Xác định termination	Quy định điều kiện kết thúc một episode.
7. Kiểm thử môi trường	Đảm bảo môi trường không lỗi, reward hợp lý và không gây leakage.
8. Huấn luyện và đánh giá Agent	Dùng thuật toán RL để train và kiểm tra hiệu quả chính sách.

- Giai đoạn thiết kế môi trường (Bước 4-6): Tiến hành lượng hóa mục tiêu thành tín hiệu phần thưởng (reward), xây dựng quy luật mô phỏng sự chuyển đổi trạng thái (transition) và xác định điều kiện kết thúc chu kỳ (termination).
- Giai đoạn thực thi (Bước 7-8): Bắt đầu bằng việc kiểm thử tính toàn vẹn của môi trường nhằm đảm bảo tính hợp lý và ngăn chặn rò rỉ dữ liệu, làm cơ sở chuẩn xác cho công tác huấn luyện và đánh giá hiệu năng chính sách của tác tử.

Appendix F Miêu tả chi tiết thực nghiệm Network

F.1 Kiến trúc Kubernetes

Để áp dụng thuật toán lập lịch thông minh cho vi dịch vụ, việc nắm vững các thành phần cốt lõi của Kubernetes là bắt buộc, vì chúng định hình trực tiếp không gian trạng thái, hành động và các ràng buộc của bài toán tối ưu:

- Pod: Đơn vị triển khai nhỏ nhất chứa các container. Trong bài toán điều phối, mỗi Pod đại diện cho một vi dịch vụ (microservice) độc lập cần được xếp lịch.
- Node: Máy chủ vật lý hoặc máy ảo cung cấp tài nguyên chạy Pod. Dung lượng của Node (CPU/RAM Allocatable) chính là các ràng buộc biên (boundary constraints) mà mô hình tối ưu phải tuân thủ để tránh sập hệ thống.
- Service: Lớp trừu tượng hóa mạng lưới, cung cấp điểm truy cập ổn định cho các Pod (vốn có tính tạm thời). Trong mô hình RL, Service giúp định hình các cạnh giao tiếp (E) và trọng số lưu lượng (w_{ij}) của đồ thị phụ thuộc.
- Deployment: Đối tượng quản lý vòng đời và trạng thái mong muốn của Pod. Đối với bộ lập lịch tự động, Deployment đóng vai trò là công cụ thực thi, chuyển đổi quyết định phân bổ của thuật toán thành các thực thể chạy trên cụm thực tế.

F.2 Quy trình triển khai kịch bản

Môi trường thực nghiệm được xây dựng dựa trên kịch bản chính 5 nodes / 8 services.

- Topology: Cụm được mô hình hóa với 5 nodes dị thể (heterogeneous), bao gồm ba nhóm cấu hình chính: small (2 cores, 4GB RAM), medium (4 cores, 8GB RAM), và large (8 cores, 16GB RAM). Việc sử dụng cụm dị thể nhằm kiểm tra khả năng ra quyết định của Agent trong việc ưu tiên các node phù hợp với tải trọng của dịch vụ.
- Service Chain: Tập hợp 8 dịch vụ đại diện cho một ứng dụng thương mại điện tử: frontend, auth, catalog, cart, recommendation, payment, shipping, database. Các dịch vụ này được thiết kế với các profile tài nguyên riêng biệt: tiny, balanced, cpu-heavy, memory-heavy, large, nhằm tạo ra sự đa dạng trong yêu cầu tài nguyên khi triển khai.

F.3 Quy trình thực nghiệm và kiểm chứng

Quá trình đánh giá được thực hiện qua hai cấp độ:

- Môi trường mô phỏng: Agent được huấn luyện qua 300.000 timesteps với cùng cấu trúc Topology.
 - Sử dụng cơ chế background load để mô phỏng tình trạng cụm đã có sẵn workload trước khi triển khai chuỗi mới, giúp Agent học cách xử lý điều kiện tài nguyên không rỗng.
- Triển khai trên cụm GKE thật:
 - Observation-aware: Ánh xạ trạng thái tài nguyên từ kubectl top nodes kết hợp với resource requests từ Pod manifest để xây dựng vector trạng thái 30 chiều.
 - Feasibility Guard: Sử dụng lớp kiểm soát an toàn trước khi kubectl apply để đảm bảo quyết định của Agent không làm quá tải node tại thời điểm thực tế, tránh các lỗi OutOfCpu hoặc OutOfMemory.
 - Đo lường: Sau khi triển khai, hệ thống sử dụng các probe Pod (với image curlimages/curl) đặt tại source services để thực hiện truy vấn HTTP thực tế, từ đó thu thập các chỉ số P50, P95 latency để đối chiếu với chi phí logic ($Cost_{latency}$) đã tối ưu trong mô phỏng.

F.4 Mô hình hóa bài toán mạng bằng MDP

Bài toán điều phối mạng lưới được chuyển đổi thành Quá trình Quyết định Markov (MDP), trong đó Agent đóng vai trò là bộ lập lịch thông minh, ra quyết định tuần tự để sắp xếp các dịch vụ lên các node mạng.

- Trạng thái (State): Môi trường mạng được biểu diễn bằng một vector trạng thái st. Kích thước của không gian trạng thái thay đổi linh hoạt theo số lượng node (m) và số lượng service (k) thông qua công thức: $\dim(s) = 4m + 2 + k$. Cụ thể, vector trạng thái bao gồm:
 - Node features: 4 thông số cho mỗi node (CPU util, RAM util, CPU capacity chuẩn hóa, RAM capacity chuẩn hóa). Để sát với thực tế triển khai, mức sử dụng hiệu dụng (effective utilization) được tính theo hướng request-aware:

$$* u_{eff}^{cpu} = \max(u_{top}^{cpu}, u_{request}^{cpu})$$

$$* u_{eff}^{mem} = \max(u_{top}^{mem}, u_{request}^{mem})$$
 - Service features: 2 thông số biểu diễn CPU và RAM request của dịch vụ hiện tại.
 - Service features: 2 thông số biểu diễn CPU và RAM request của dịch vụ hiện tại.

- **Hành động (Action):** Không gian hành động là một tập hợp rời rạc các node khả dụng. Tại mỗi bước, hành động a_t là việc Agent chọn 1 node n_i để đặt dịch vụ: $A=0, 1, \dots, m-1$ (Ví dụ: Với cụm 5 nodes, không gian hành động là $A = 0, 1, 2, 3, 4$).
 - **Phần thưởng (Reward):** Hàm phần thưởng được thiết kế đa mục tiêu để cân bằng giữa tính khả thi và hiệu năng. Hàm phần thưởng tổng quát được định nghĩa là:
 - Thưởng điểm cơ bản khi cấp phát thành công ($R_{\text{base}} = 5$).
 - Phạt cực nặng nếu Agent chọn node không tồn tại ($R = -50$) hoặc node không đủ tài nguyên gây lỗi cấp phát ($R = -20$).
 - Các điểm phạt phụ (P) được áp dụng nếu gây quá tải (overload), làm mất cân bằng tải (imbalance) giữa các node, hoặc tạo điểm nóng cục bộ (hotspot).
 - Trong kiến trúc mạng vi dịch vụ, mỗi ứng dụng được mô hình hoá dưới dạng 1 đồ thị có hướng: $G=(S,E)$ Trong đó:
 - * S là tập các dịch vụ
 - * E là tập các cạnh phụ thuộc
 - * Mỗi cạnh $(s_i, s_j) \in E$ sở hữu 1 trọng số lưu lượng w_{ij} , đại diện cho mức độ quan trọng và tần suất giao tiếp giữa 2 dịch vụ
 - Phạt dựa trên độ trễ trọng số giao thông (traffic-weighted latency penalty) với ma trận độ trễ $L \in R^{m \times m}$, L_{pq} là độ trễ vật lý giữa node n_p và node n_q . Chi phí độ trễ của mạng được tính theo công thức: $Cost_{\text{latency}} = \sum_{(s_i, s_j) \in E} w_{ij} \cdot L_{f(s_i), f(s_j)}$ Trong đó: Hàm ánh xạ $f(s_i)$ trả về node mà dịch vụ s_i được cấp phát
- Agent buộc phải học cách đặt các dịch vụ có w_{ij} cao vào cùng 1 node hoặc các node có độ trễ L tiệm cận 0 để tối ưu hoá hiệu năng mạng toàn cục.

F.5 Triển khai và đánh giá thuật toán DRL

Để giải quyết bài toán không gian trạng thái phức tạp của hệ thống mạng, các thuật toán Học tăng cường sâu (Deep Reinforcement Learning) thường được áp dụng:

- **Deep Q-Network (DQN)**
 - Khái niệm: Phương pháp sử dụng mạng nơ-ron sâu (deep neural network) để ước tính các giá trị Q-value thay vì sử dụng 1 bảng lưu trữ (Q-table) truyền thống. Các giá trị Q-value đại diện cho mức độ hiệu quả của 1 hành động cụ thể khi agent đang ở trong 1 trạng thái nhất định.
 - Ưu điểm: Việc sử dụng mạng nơ-ron giúp DQN khắc phục sự chồng chéo của Q-Learning truyền thống, cho phép agent đưa ra quyết định trong các môi trường có không gian trạng thái rất lớn (high-dimensional state spaces) hoặc dữ liệu đầu vào mang tính liên tục như hình ảnh video.
 - Các thành phần cốt lõi:
 - * Experience Replay: Mô hình lưu trữ các trải nghiệm trong quá khứ (s, a, r, s') vào bộ đệm D và lấy mẫu ngẫu nhiên trong quá trình huấn luyện nhằm phá vỡ sự tương quan của dữ liệu.
 - * Target Network: Tương tự Q-network chính nhưng được cập nhật trọng số ít thường xuyên hơn, giúp cung cấp các mục tiêu Q-value ổn định. Giá trị

- mục tiêu (Target Y) được tính toán bằng phương trình Bellman: $Y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ (Trong đó: r là phần thưởng nhận được, là hệ số chiết khấu, - là bộ trọng số của mạng mục tiêu)
- * Loss Function (Hàm mất mát): Thường sử dụng sai số bình phương trung bình (MSE) để tối thiểu hoá khoảng cách giữa giá trị Q-value dự đoán và mục tiêu: $L(\theta) = E_{(s,a,r,s') \sim D} [(Y - Q(s, a; \theta))^2]$ (Với θ là trọng số của mạng mục tiêu)
 - Rất phù hợp với các bài toán có không gian hành động rời rạc (chọn 1 trong N node mạng). DQN sử dụng mạng nơ-ron để xấp xỉ hàm giá trị, giúp quyết định node mạng nào mang lại lợi ích lâu dài cao nhất.
- Advantage Actor-Critic (A2C)
 - Thuật toán sử dụng kiến trúc kết hợp gồm 2 thành phần: 1 Actor chuyên làm nhiệm vụ chọn hành động (cập nhật chính sách) và 1 Critic chuyên đánh giá các hành động đó (cập nhật giá trị trạng thái V). Cấu trúc này giúp giảm phương sai của policy gradients và xử lý hiệu quả bài toán đánh đổi giữa phương sai và độ chệch (bias/variance tradeoff)
 - Cơ chế hàm lợi thế (Advantage Function)
 - * Hàm lợi thế $A(s, a)$ được dùng để đo lường mức phần thưởng tăng thêm so với giá trị trung bình kỳ vọng mà agent có thể nhận được. Nó được định nghĩa toán học là: $A(s, a) = Q(s, a) - V(s)$
 - * A2C thường ước lượng lợi thế này bằng sai số Temporal Difference (TD error) $\hat{A}_t \approx r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
 - * Quá trình huấn luyện diễn ra song song với 2 hàm mục tiêu:
 - Dành cho Actor (Tối đa hoá lợi thế): Mạng Actor được cập nhật dựa trên gradient của hàm log-xác suất nhân với lợi thế $\Delta_\theta J(\theta) = E[\Delta_\theta \log \pi_\theta(a_t|s_t) \hat{A}_t]$
 - Dành cho Critic (Giảm thiểu sai số): Mạng critic cố gắng đưa hàm giá trị V tiến gần đến giá trị mục tiêu bằng hàm mất mát MSE: $L_v(\Phi) = E[(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2]$
 - Ý nghĩa: Nếu $A_t > 0$, hành động hiện tại mang lại kết quả tốt hơn mức trung bình, gradient sẽ đẩy Actor ưu tiên thực hiện hành động này nhiều hơn. Ngược lại, gradient sẽ kìm hãm việc lựa chọn hành động đó - Proximal Policy Optimization (PPO) Khái niệm: PPO là 1 thuật toán học tăng cường giúp agent liên tục cải thiện hành động của mình trong khi vẫn duy trì được sự ổn định trong quá trình huấn luyện. Cơ chế hoạt động:
 - PPO cập nhật chính sách của agent thông qua policy gradients. Để đo lường mức độ thay đổi của chính sách, thuật toán sử dụng tỷ lệ xác suất (probability ratio) giữa chính sách mới và chính sách cũ: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
 - Thay vì tối đa hoá trực tiếp các phần thưởng, PPO tiến hành tối đa hoá 1 hàm mục tiêu đại diện (surrogate objective). Sử dụng 1 quy tắc cắt xén (clipping rule) đặc biệt để giới hạn các thay đổi quá lớn gây mất ổn định: $L^{CLIP}(\theta) = \hat{E}_t[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$ (Trong đó: $\hat{A}_t$ là ước lượng hàm lợi thế, là siêu tham số giới hạn phạm vi cập nhật - thường được đặt là 0.2)

- Proximal Policy Optimization (PPO) & Advantage Actor-Critic (A2C): Các phương pháp dựa trên tối ưu hóa chính sách (Policy Gradient) giúp Agent học trực tiếp xác suất chọn node, duy trì sự ổn định trong các hệ thống mạng có quy mô thay đổi linh hoạt.

F.6 Đánh giá hiệu quả

Thực nghiệm cho thấy các mô hình mạng được điều khiển bằng RL (đặc biệt là DQN) vượt trội hơn hẳn các thuật toán heuristic truyền thống như Random, First-Fit hay Round-Robin. Agent có khả năng tự động nhận diện các dịch vụ có liên kết mạng mật thiết và đặt chúng trên cùng một node (co-location) để giảm độ trễ, nhưng vẫn giữ được tỷ lệ phân tán hợp lý để tránh tạo ra hotspot, đảm bảo tính ổn định bền vững cho toàn bộ hạ tầng mạng. Biểu đồ hình 1 thể hiện sự hội tụ của ba thuật toán DQN, PPO và A2C qua 300.000 bước huấn luyện (tương ứng khoảng 37.500 episodes). Quá trình huấn luyện trải qua ba giai đoạn chính:

- Giai đoạn khám phá (0 - 5.000 episodes): Cả ba thuật toán đều ghi nhận mức reward thấp và biến động mạnh. Trong giai đoạn này, các Agent chủ yếu thực hiện hành động ngẫu nhiên, dẫn đến việc thường xuyên vi phạm các ràng buộc tài nguyên và chịu phạt từ hàm reward.
- Giai đoạn học tập (5.000 - 15.000 episodes): Reward trung bình tăng trưởng nhanh chóng, cho thấy các Agent đã nắm bắt được quy tắc hệ thống: ưu tiên các node còn tài nguyên và hạn chế các hành động gây quá tải.
- Giai đoạn ổn định (sau 20.000 episodes): Các thuật toán dần hội tụ về mức reward tối ưu. DQN đạt mức reward ổn định và cao nhất, trong khi PPO và A2C hội tụ ở ngưỡng thấp hơn.

So sánh hiệu năng:

- DQN: Thể hiện ưu thế vượt trội với mức reward trung bình tốt nhất (tiệm cận ngưỡng -10 đến -20). Sự ổn định này phản ánh tính hiệu quả của cơ chế Value-based trong không gian hành động rời rạc của bài toán Microservice Chain Placement, nơi việc gán giá trị trực tiếp cho từng node (index) mang lại độ chính xác cao.
- PPO và A2C: Cả hai thuật toán đều hội tụ ở mức reward thấp hơn (dao động quanh vùng -40 đến -50). Mặc dù PPO thường ổn định trong các bài toán điều khiển liên tục, kết quả cho thấy thuật toán này cần điều chỉnh thêm về siêu tham số (hyperparameters) để tối ưu hóa trong không gian hành động rời rạc của đề án.

Kết luận: Dựa trên kết quả thực nghiệm, DQN được lựa chọn làm mô hình chính cho việc triển khai trên cụm Kubernetes thật (GKE). Khả năng ánh xạ trực tiếp từ observation vector (30 chiều) sang giá trị Q-value của các node giúp DQN giải quyết bài toán đa mục tiêu—vừa đảm bảo khả năng triển khai thành công, vừa tối ưu hóa phân bổ tải và hạn chế hiện tượng điểm nóng (hotspot) một cách hiệu quả.

Appendix G Minh họa quá trình ra quyết định bằng Reinforcement Learning trong kinh tế cá nhân

G.1 Quan sát trạng thái tài chính

Để hiểu cách RL hoạt động trong bối cảnh kinh tế, hãy theo dõi một ví dụ cụ thể: một tài xế gig-economy quyết định chi tiêu mỗi tuần.

Tại đầu tuần t , tài xế quan sát trạng thái hiện tại: “Tôi có \$5,000 trong tài khoản. Trong tuần vừa rồi, tôi kiếm được trung bình \$800. Tôi chưa biết chắc tuần này sẽ kiếm được bao nhiêu, nhưng dựa trên dữ liệu lịch sử, tôi dự kiến khoảng \$750 đến \$900. Có một chính sách thuế mới vừa có hiệu lực”. Toàn bộ thông tin này tạo nên trạng thái:

$$s_t = (x_t, \mathbb{E}[y_{t+1}], \text{policy_info}).$$

G.2 Lựa chọn hành động

Dựa trên trạng thái hiện tại, tài xế quyết định: “Tuần này, tôi sẽ chi tiêu \$400 cho thực phẩm, xăng và các nhu cầu thiết yếu, đồng thời dành lại \$200 làm quỹ dự phòng. Phần còn lại sẽ được bù đắp bằng thu nhập từ các chuyến xe trong tuần.” Hành động này được biểu diễn là:

$$a_t = c_t = 400.$$

Lựa chọn này xuất phát từ policy mà tài xế đã hình thành qua nhiều tuần kinh nghiệm. Nói cách khác, trong những trạng thái tương tự trước đây, anh ta đã học được mức chi tiêu nào thường mang lại kết quả tốt nhất.

G.3 Nhận Reward và quan sát trạng thái mới

Sau khi thực hiện quyết định chi tiêu, môi trường kinh tế phản hồi thông qua phần thưởng và trạng thái mới.

Reward tức thời được tạo thành từ lợi ích của việc tiêu dùng và mức độ an tâm khi vẫn duy trì được khoản tiền dự phòng:

$$R_t = u(400) - \text{anxiety_cost}(\text{low_savings}).$$

Tuy nhiên, môi trường luôn chứa yếu tố bất định. Trong tuần này, chiếc xe bất ngờ bị hỏng khiến tài xế phải chi thêm \$150 để sửa chữa. Đồng thời, do thời tiết xấu và mức độ cạnh tranh tăng cao, thu nhập thực tế chỉ đạt \$600 thay vì khoảng \$750–\$900 như dự kiến.

Do đó, trạng thái tài chính ở tuần tiếp theo được cập nhật theo phương trình:

$$x_{t+1} = r(x_t - c_t) + y_{t+1} - \text{shock},$$

với $r = 1.001$, tương ứng:

$$x_{t+1} = 1.001(5000 - 400) + 600 - 150 \approx 5050.$$

G.4 Cập nhật kiến thức và học hỏi

Từ kết quả của tuần này, tài xế điều chỉnh đánh giá của mình về quyết định chi tiêu \$400 trong trạng thái tài chính hiện tại. Reward vẫn mang giá trị tích cực vì anh ta duy trì được khả năng thanh toán và vẫn còn khoản tiền dự phòng. Tuy nhiên, mức Reward thấp hơn kỳ vọng do ảnh hưởng của cú sốc bất ngờ.

Theo cơ chế học của Reinforcement Learning, Agent sẽ dần học được rằng khi có khoảng \$5,000 tài sản và mức thu nhập kỳ vọng khoảng \$700–\$800 mỗi tuần, việc chi tiêu \$400 tương đối an toàn nhưng chưa chắc là tối ưu. Trong những lần ra quyết định tiếp theo, Agent có thể lựa chọn tăng mức tiêu dùng để cải thiện chất lượng cuộc sống hoặc tiết kiệm nhiều hơn nhằm giảm rủi ro trước các cú sốc tài chính.

G.5 Lặp lại quá trình học

Ở tuần kế tiếp, tài xế tiếp tục quan sát trạng thái mới, đưa ra quyết định chi tiêu, nhận Reward và cập nhật chính sách. Quá trình này lặp lại qua hàng chục hoặc hàng trăm tuần. Theo thời gian, policy sẽ hội tụ về một chiến lược ổn định nhằm tối đa hóa tổng lợi ích kỳ vọng trong dài hạn:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t R_t \right],$$

trong đó β là hệ số chiết khấu phản ánh việc Agent ưu tiên lợi ích hiện tại hơn lợi ích trong tương lai.

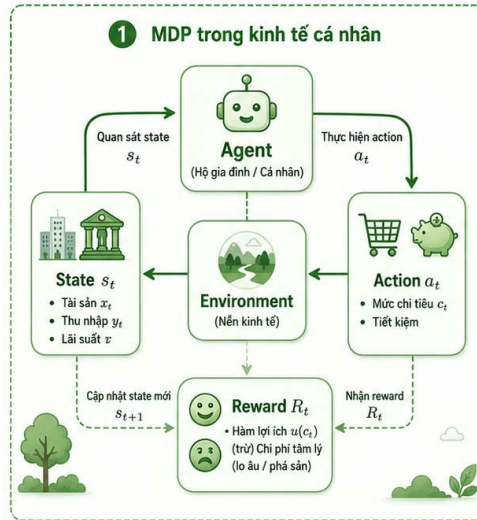


Figure G3: Minh họa quá trình ra quyết định bằng Reinforcement Learning

Appendix H Phân tích toàn diện về động lực học tài sản và chỉ số bất bình qua các kịch bản mô phỏng

H.1 Phân tích trực quan và mô tả dữ liệu định lượng

Hệ thống mô phỏng dựa trên tác tử (Agent-Based Modeling) kết hợp ngẫu nhiên Monte Carlo đã tái hiện rõ nét sự phân hóa tài sản và quỹ đạo tâm lý-kinh tế của các nhóm lao động trong điều kiện rủi ro. Việc quan sát trực quan các đường quỹ đạo này cho phép bóc tách những đứt gãy cấu trúc mà các phương pháp thống kê trung bình tính thường bỏ sót.

H.1.1 Kịch bản 1: Chênh lệch thu nhập nhỏ (Minor Income Difference)

Kịch bản này thiết lập trạng thái nền với hai tác tử xuất phát từ cùng một vạch xuất phát tài sản ban đầu ($X_0 = 45$). Tuy nhiên, cấu trúc nội tại về dòng thu nhập tạo ra một sự phân kỳ (divergence) sâu sắc ở cuối chu kỳ chuỗi thời gian ($t = 60$).

Quỹ đạo tài sản. Tác tử 1 (Agent 1, nhãn = 1) đại diện cho nhóm có khả năng chống chịu tốt hơn. Dù trải qua giai đoạn suy giảm tài sản trong một phần ba đầu chu kỳ (rơi từ 45 xuống ngưỡng đáy ≈ 20 tại thời điểm $t \approx 20$), tác tử này nhanh chóng kích hoạt chiến lược tái cấu trúc chỉ tiêu để phục hồi mạnh mẽ về mức 37 tài sản khi chạm mốc $t = 60$. Ngược lại, Tác tử 2 (Agent 2, nhãn = 0) phản ánh một quỹ đạo tổn thương nghiêm trọng. Sau khi sụt giảm, tài sản của Agent 2 lao dốc không phanh và rơi thẳng về vùng cận kiệt (≈ 3 tài sản tại $t = 60$). Khoảng cách tài sản cuối kỳ lên tới **34 đơn vị** (37 so với 3) chứng minh rằng một biên độ lệch nhỏ về thu nhập ban đầu có thể bị khuếch đại thành một hố sâu phân hóa tài sản khổng lồ dưới tác động của các cú sốc ngẫu nhiên.

Động lực học bấp bênh (Precarity Index). Sự phân hóa tài sản đồng quy với diễn biến của chỉ số bấp bênh. Tâm lý bấp bênh của Agent 1 đạt đỉnh (peak) ở mức ≈ 2.5 trong giai đoạn khủng hoảng tài sản ($t \approx 20$) nhưng sau đó hạ nhiệt nhanh chóng và ổn định ở mức thấp ≈ 0.9 , cho thấy sự giải tỏa áp lực tài chính. Đối với Agent 2, chỉ số bấp bênh không chỉ tăng vọt lên mức đỉnh cao hơn (≈ 3.2) mà còn bị “khóa cứng” (trapped) tại mức đỉnh này cho đến cuối kỳ. Điều này minh chứng cho trạng thái “tư thế phòng vệ vĩnh viễn” (permanent defensive posture) về mặt kinh tế, nơi tác tử hoàn toàn mất đi khả năng lập kế hoạch dài hạn.

H.1.2 Kịch bản 2: Can thiệp thuế (Tax Intervention)

Kịch bản 2 là một thực nghiệm động lực học chính sách nhằm tìm kiếm thời điểm (timing) tối ưu để can thiệp tài khóa vào hệ thống nhằm cứu vãn các tác tử đang trên bờ vực phá sản.

Quỹ đạo tài sản. Nếu hoàn toàn không có can thiệp (No intervention), quỹ đạo của tác tử yếu thế sẽ bị triệt tiêu hoàn toàn về mức đáy ≈ 5 tại $t = 60$. Khi áp dụng các kịch bản can thiệp thuế để điều tiết và trợ cấp, kết quả thay đổi rõ rệt theo thời gian. Can thiệp sớm (Early tax) giúp ổn định quỹ đạo ngay từ giữa chu kỳ, tài sản không bị rơi quá sâu (giữ quanh mức 20) và phục hồi nhẹ lên 22. Điểm bắt ngờ mang tính kịch tính lớn nhất nằm ở kịch bản can thiệp muộn (Late tax): tài sản của tác tử được phục hồi ngoạn mục lên mức cao nhất hệ thống (≈ 40 tại $t = 60$).

Diễn biến của chỉ số bấp bênh. Can thiệp sớm tạo ra một lộ trình bằng phẳng, chỉ số bấp bênh dao động ổn định trong khoảng 1.5–2.0 và có phương sai rất bé. Trong khi đó, can thiệp muộn chấp nhận một sự đánh đổi nguy hiểm: chỉ số bấp bênh bị đẩy lên mức cực đoan (≈ 3.2 – 3.3) và tạo ra một đỉnh cấp tính (acute peak) tăng vọt đột ngột tại thời điểm $t \approx 25$. Ngay sau khi chính sách can thiệp muộn được kích hoạt, chỉ số này lao dốc thẳng đứng xuống mức an toàn nhất toàn hệ thống (≈ 1.0). Điều này gợi mở một insight sâu sắc: việc trì hoãn can thiệp tạo điều kiện cho tài sản tích lũy mạnh hơn ở giai đoạn sau, nhưng buộc hệ thống phải chịu đựng một giai đoạn “sốc áp lực” khủng hoảng tâm lý cực hạn ở giai đoạn giữa.

H.1.3 Kịch bản 3: Lao động tạm thời so với lao động văn phòng (Gig worker vs Office worker)

Đây là kịch bản đại diện cho sự phân hóa mang tính cấu trúc sâu sắc nhất trong nền kinh tế hiện đại, nơi hai nhóm đối tượng bắt đầu với mức tích lũy tài sản ban đầu bằng nhau nhưng rất cao ($X_0 = 200$).

Quỹ đạo tài sản. Nhóm lao động văn phòng (Office Worker) với dòng thu nhập ổn định thể hiện một đường cong trơn, giảm có kiểm soát do chi tiêu tiêu dùng cố định và ổn định vững chắc ở mức 95–100 tài sản. Ngược lại, nhóm lao động tự do (Gig Worker) đối mặt với một quỹ đạo phân rã tự do. Từ mức 200, tài sản của họ bị bào mòn nhanh chóng xuống 65 ở giai đoạn $t \approx 25$ và tiếp tục phân rã hoàn toàn về mức tiệm cận phá sản (≈ 5) khi kết thúc chu kỳ. Khoảng cách tài sản cuối kỳ chạm ngưỡng **95 đơn vị** (100 so với 5), khẳng định đây là kịch bản phân hóa cực đoan nhất trong toàn bộ nghiên cứu.

Động lực học bấp bênh. Nhóm lao động văn phòng duy trì chỉ số bấp bênh ở mức lý tưởng (0.3–0.4) xuyên suốt chu kỳ. Đối với Gig worker, chỉ số bấp bênh bật tăng mạnh mẽ từ 0.3 lên 2.1 ngay khi tài sản bị tổn hao 30% (tại $t \approx 25$) và bị đóng băng hoàn toàn ở mức cao này. Hệ quả này chỉ ra rằng lao động Gig không chỉ tổn thương về mặt vật chất mà còn rơi vào một trạng thái khủng hoảng kinh tế cấu trúc kinh niên không thể tự thoát ra nếu thiếu lực đẩy từ bên ngoài.

H.2 So sánh chỉ số và cơ chế động lực học hành vi

Để luận giải các kết quả trực quan trên, nghiên cứu tiến hành lượng hóa rủi ro thông qua xác suất phá sản tích lũy và phân tích các cơ chế vi mô dựa trên lý thuyết kinh tế học hành vi.

H.2.1 Ước lượng xác suất phá sản tích lũy (Cumulative Ruin Probability)

Xác suất phá sản được định nghĩa là xác suất mà quỹ đạo tài sản của một tác tử chạm vào hoặc rơi xuống dưới ngưỡng hấp thụ bằng 0 (absorbing barrier) tại bất kỳ thời điểm nào trong chu kỳ mô phỏng.

Nhóm nguy cơ cực đoan (Ruin Probability > 80%). Gig Worker trong Kịch bản 3 giữ tỷ lệ phá sản cao nhất toàn hệ thống ($\approx 95\%$), với thời gian chạm ngưỡng khủng hoảng rất sớm ($t \approx 35\text{--}40$). Xếp ngay sau là Agent 2 trong Kịch bản 1 ($\approx 85\text{--}90\%$) và trường hợp không can thiệp của Kịch bản 2 ($\approx 80\%$). Điểm chung của nhóm này là sự thiếu vắng các cơ chế phòng vệ chống lại các biến động có phương sai lớn.

Nhóm an toàn cấu trúc (Ruin Probability < 30%). Can thiệp thuế sớm giúp kéo giảm xác suất phá sản xuống còn $\approx 25\text{--}30\%$. Kết quả tối ưu nhất thuộc về nhóm can thiệp muộn ($\approx 5\text{--}10\%$) và lao động văn phòng ($\approx 10\text{--}15\%$). Điều này chứng minh rằng sự ổn định của dòng thu nhập hoặc một cú hích chính sách đúng thời điểm có thể đảo chiều hoàn toàn xác suất đổ vỡ kinh tế của hộ gia đình.

H.2.2 Phân tích căn nguyên vi mô của sự khác biệt

A. Tác động phi tuyến của tính e ngại rủi ro và hữu dụng biên giảm dần (DMU). Nền tảng hành vi của các tác tử được dẫn dắt bởi hàm hữu dụng CRRA:

$$U(c) = \frac{c^{1-\gamma}}{1-\gamma} \quad (\text{H1})$$

trong đó γ phản ánh hệ số e ngại rủi ro tương đối hằng số.

Cơ chế này giải thích tại sao Agent 2 trong Kịch bản 1, dù có mức thu nhập không quá thấp, vẫn rơi vào **“Bẫy bấp bênh” (Precarity Trap)**. Dưới tác động của quy luật hữu dụng biên giảm dần, khi tài sản của Agent 2 bắt đầu suy giảm do các cú sốc ngẫu nhiên ban đầu, hữu dụng biên của việc tiêu dùng hiện tại tăng vọt lên mức rất cao. Để bảo vệ mức tiêu dùng tối thiểu nhằm duy trì hữu dụng, Agent 2 buộc phải thực hiện các khoản chi tiêu mang tính phòng vệ hoảng loạn. Hành vi ưu tiên tiêu dùng ngắn hạn này triệt tiêu hoàn toàn khả năng tích lũy vốn cho tương lai. Tác tử bị mắc kẹt trong vòng lặp: tài sản giảm $\rightarrow$ hữu dụng biên tăng $\rightarrow$ kích hoạt chi tiêu phòng vệ tuyệt vọng $\rightarrow$ không thể tích lũy tài sản $\rightarrow$ phá sản.

B. Cơ chế bào mòn tài sản dự trữ (Reserve Asset Depletion). Cơ chế này vận hành rõ ràng và tàn nhẫn nhất trong trường hợp của Gig Worker (Kịch bản 3). Do đặc thù thu nhập của kinh tế chia sẻ phụ thuộc vào thuật toán nền tảng, dòng thu nhập của họ có phương sai (variance) cực lớn. Để đạt được trạng thái san phẳng tiêu dùng (consumption smoothing), Gig Worker đòi hỏi một bộ đệm tài sản dự trữ (x_t) lớn hơn gấp nhiều lần so với lao động truyền thống.

Khi các cú sốc âm liên tục xảy ra, tài sản dự trữ đóng vai trò là vùng đệm hấp thụ xung lực. Tuy nhiên, do tốc độ và tần suất của các cú sốc ngẫu nhiên vượt quá khả năng tái tạo thu nhập, bộ đệm này bị xói mòn liên tục. Khi tài sản dự trữ chạm ngưỡng tới hạn (tại $t \approx 25$), năng lực tự phục hồi hoàn toàn biến mất. Tài sản chuyển sang trạng thái “chảy máu cấu trúc”, sụt giảm tuyến tính thẳng về ngưỡng 0.

Ngược lại, hiện tượng này được minh chứng ở chiều tích cực tại Kịch bản 2 (Can thiệp muộn). Việc trì hoãn can thiệp vô tình tạo ra một khoảng thời gian trống buộc tác tử phải tự thắt chặt hành vi chi tiêu một cách nghiêm ngặt. Khi chính sách thuế và trợ cấp được bơm vào tại mốc $t \approx 25$, nó tạo ra một “cú sốc tích cực” (positive shock). Nhờ bộ đệm hành vi đã được siết chặt trước đó, tác tử tận dụng toàn bộ dòng tiền mới để tái lập tài sản tích lũy thay vì tiêu dùng hoảng loạn, giúp tài sản bật tăng mạnh mẽ lên mức 40 đơn vị.

H.3 Hàm ý chính sách chuyên sâu

Từ các phát hiện mang tính định lượng của mô hình, nghiên cứu đề xuất ba trục kiến nghị chính sách cốt lõi nhằm tái cấu trúc lưới an sinh xã hội trong nền kinh tế số.

Hàm ý 1: Triển khai lưới an sinh tài chính “Sớm và Chủ động” (Proactive Co-insurance). Kịch bản 3 chỉ ra rủi ro phá sản của lao động Gig cao gấp gần 10 lần lao động văn phòng (95% so với 10%), và Kịch bản 2 khẳng định can thiệp sớm kiểm soát hoàn toàn phương sai của chỉ số bấp bênh, giữ nó không vượt quá ngưỡng 2.0. Do đó, cần chấm dứt tư duy cứu trợ tính (chỉ hỗ trợ khi khủng hoảng đã xảy ra) và thay vào đó luật hóa cơ chế **Bảo hiểm thất nghiệp tích hợp nền tảng** cho lao động Gig, yêu cầu các nghiệp đoàn hoặc ứng dụng công nghệ tự động trích 2%–3% doanh thu trên mỗi chuyến hàng/đơn hàng vào một Tài khoản An toàn Xã hội cá nhân. Cơ chế đồng bảo hiểm (co-insurance) này đóng vai trò như một bộ đệm tài sản được tích lũy chủ động từ sớm, ngăn chặn việc tác tử rơi vào bẫy bấp bênh ngay từ những chu kỳ đầu tiên của thị trường.

Hàm ý 2: Chính sách thuế động linh hoạt “Thích ứng theo chu kỳ tài sản” (Adaptive Fiscal Policy). Kịch bản 2 tiết lộ một nghịch lý kinh tế: can thiệp muộn tối ưu hóa tài sản tích lũy cuối kỳ tốt hơn can thiệp sớm (40 so với 22) nhưng lại để lại một đỉnh bấp bênh cấp tính nguy hiểm ở giai đoạn giữa chu kỳ. Hệ thống tài khóa cần chuyển đổi sang mô hình **Thuế tiêu dùng động thích ứng** (Adaptive VAT/Consumption Tax) dựa trên dữ liệu thời gian thực:

(i) trong *giai đoạn tích lũy an toàn*, khi thu nhập và tài sản của quần thể lao động Gig tăng trưởng ổn định, áp dụng mức thuế tiêu dùng tiêu chuẩn để tích lũy ngân sách công; (ii) trong *giai đoạn suy thoái rủi ro*, khi hệ thống ghi nhận các cú sốc thị trường, lập tức giảm thuế tiêu dùng và chuyển dịch ngân sách sang hỗ trợ tiền mặt trực tiếp nhằm “bẻ gãy” đỉnh bấp bênh cấp tính tại các thời điểm nhạy cảm, vừa đảm bảo an toàn tâm lý kinh tế, vừa bảo lưu năng lực tích lũy tài sản dài hạn cho người dân.

Hàm ý 3: Cơ chế phân bổ rủi ro hai mức nhọn (Dual-level Risk Management). Kịch bản 1 và 3 chứng minh rằng ngay cả khi chênh lệch thu nhập ban đầu là rất nhỏ hoặc tài sản ban đầu rất cao, rủi ro cấu trúc và tính biến động của thị trường vẫn có khả năng đẩy các hộ gia đình vào trạng thái khánh kiệt hoàn toàn – một lỗi đổ vỡ thuộc về cấu trúc hệ thống, không phải do năng lực cá nhân. Giải pháp đề xuất gồm hai mức độ: (i) *mức độ vi mô* – Tài khoản bảo mật kỹ thuật số (Digital Security Account) với các tài khoản tiết kiệm bị “khóa cứng” (locked-in savings) trên ứng dụng lao động, tự động trích một phần thặng dư thu nhập trong những ngày cao điểm để cường bức hành vi tiết kiệm, chống lại áp lực của hàm hữu dụng biên tăng cao khi có sốc; (ii) *mức độ vĩ mô* – Quỹ ổn định kinh tế chia sẻ (Gig Economy Stabilization Fund) được tài trợ bằng thuế đóng góp từ các tập đoàn công nghệ nắm giữ nền tảng, với mục tiêu định lượng kéo giảm xác suất phá sản cấu trúc của Gig worker từ 95% xuống vùng an toàn 30%–40%, đồng thời kéo chỉ số bấp bênh kinh niên từ mức 2.1 về mức tiệm cận lao động truyền thống (≈ 1.2).

H.4 Bảng tổng hợp và đối chiếu hệ thống mô phỏng

Table H5: Tổng hợp và đối chiếu định lượng ba kịch bản mô phỏng.

Thước đo định lượng	KB1: Chênh lệch thu nhập nhỏ	KB2: Can thiệp thuế chính sách	KB3: Gig vs Office Worker
Xác suất phá sản tích lũy (nhóm yếu thế/Gig)	$\approx 85\%$ – 90%	$\approx 80\%$ (không can thiệp)	$\approx 95\%$ (nguy hiểm nhất)
Xác suất phá sản tích lũy (nhóm ưu thế/Office)	$\approx 15\%$ – 20%	$\approx 5\%$ – 10% (can thiệp muộn)	$\approx 10\%$ – 15% (an toàn bền vững)
Đỉnh chỉ số bấp bênh (Precarity Peak)	3.2 (Agent 2 – mắc kẹt kinh niên)	2.5 (đỉnh cấp tính, can thiệp muộn)	2.1 (Gig – đóng băng vùng nguy hiểm)
Khoảng cách tài sản cuối kỳ ($t = 60$)	37 vs 3 (≈ 34)	40 vs 5 (≈ 35)	100 vs 5 (≈ 95 , phân hóa cực đoan)
Tốc độ bào mòn tài sản dự trữ	Trung bình – áp lực chi tiêu phòng vệ	Biến động mạnh – phụ thuộc timing chính sách	Rất nhanh và nghiêm trọng – phương sai thu nhập lớn
Bản chất đứt gãy kinh tế	Bẫy bấp bênh tâm lý hành vi do hữu dụng biên	Sai lệch thời điểm điều tiết của cơ quan quản lý	Đứt gãy cấu trúc hệ thống kinh tế số
Khuyến nghị chiến lược cốt lõi	Tiết kiệm vi mô bắt buộc	Thuế động thích ứng theo chu kỳ tài sản	Lưới an sinh chủ động + quỹ ổn định vĩ mô

H.5 Kết luận phụ lục

Qua phân tích chi tiết ba kịch bản mô phỏng, nghiên cứu rút ra ba kết luận khoa học mang tính nền tảng:

1. Kịch bản 3 phản ánh thực trạng khốc liệt nhất của nền kinh tế chia sẻ. Nếu không có các can thiệp ngoại ứng, sự tương tác giữa phương sai thu nhập lớn và đặc tính phi tuyến của hàm hữu dụng CRRA chắc chắn sẽ đẩy lao động tự do vào trạng thái khánh kiệt hoàn toàn, bất kể mức tích lũy ban đầu của họ lớn ra sao.
2. Can thiệp tài khóa có vai trò đảo chiều quỹ đạo phá sản, nhưng yếu tố quyết định không nằm ở quy mô dòng tiền mà nằm ở **thời điểm can thiệp (timing)**. Việc phối hợp giữa can thiệp sớm để ổn định tâm lý và can thiệp muộn để tích lũy tài sản là chìa khóa để tối ưu hóa chính sách.
3. Sự tồn tại của “Bẫy bấp bênh vô hình” trong Kịch bản 1 cảnh báo rằng việc chỉ gia tăng thu nhập thuần túy cho người lao động là không đủ. Nếu thiếu các định chế chính sách buộc tiết kiệm và quản trị rủi ro ngẫu nhiên, hành vi chi tiêu phòng vệ mang tính bản năng của con người sẽ tự động triệt tiêu năng lực tích lũy, khiến họ mãi mãi mắc kẹt dưới ngưỡng tổn thương kinh tế.

Appendix I Các kiến trúc World Model mà nhóm đề xuất thực nghiệm

I.1 World Models – Ha & Schmidhuber 2018

Kiến trúc này gồm ba phần chính: Vision Model, Memory Model và Controller. Vision Model dùng VAE để nén ảnh thành latent vector, Memory Model dùng MDN-RNN để học động lực trong latent space, còn Controller sinh hành động từ latent state. Điểm mạnh là Agent ra quyết định trên biểu diễn latent thay vì ảnh gốc, nhưng Controller thường được tối ưu bằng CMA-ES, chưa học end-to-end bằng gradient.

I.2 PlaNet

PlaNet học latent dynamics từ ảnh pixel và dùng mô hình này để lập kế hoạch hành động. Thành phần chính là RSSM, kết hợp trạng thái tắt định và xác suất để dự đoán latent state, reward và quan sát tương lai. Khi chọn hành động, PlaNet dùng CEM để thử nhiều chuỗi hành động trong latent space. Ưu điểm là planning trực tiếp từ ảnh, nhưng chi phí tính toán cao do phải planning online ở mỗi bước.

I.3 DreamerV1

DreamerV1 kế thừa RSSM từ PlaNet nhưng thay online planning bằng học Actor-Critic trong latent space. World Model trước tiên học biểu diễn môi trường, sau đó Actor và Critic được huấn luyện qua các imagination rollout. Nhờ học từ trajectory tưởng tượng, Dreamer sử dụng dữ liệu hiệu quả hơn và có thể hành động trực tiếp bằng Actor sau khi huấn luyện.

I.4 Transformer World Model – IRIS

IRIS dùng discrete autoencoder để biến ảnh thành token rời rạc, sau đó dùng autoregressive Transformer để dự đoán chuỗi token tương lai. Cách tiếp cận này xem dynamics như bài toán sequence modeling. IRIS tận dụng khả năng mô hình hóa chuỗi mạnh của Transformer, phù hợp với dữ liệu hình ảnh phức tạp, nhưng đòi hỏi tài nguyên tính toán lớn và thiết kế tokenizer cẩn thận.